

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ



**НГТУ
НЭТИ**

**Лабораторная работа №2
по дисциплине «ТРПО-2»
на тему
«Разработка класса структуры данных на Kotlin»**

Группа:

Факультет: АВТФ

Студенты:

Преподаватель: Романов Е.Л.

Новосибирск 2022

Оглавление

1 Задание на лабораторную работу	2
2 Исходные данные	3
3 Ход работы.....	3
3.1 Типы данных	3
3.2 Паттерны/шаблоны проектирования	3
3.3.1. Фабрика (Factory).....	3
3.3.2. Объект-прототип/Строитель (Prototype/Builder)	4
3.3 Интерфейсы	5
3.4 Структура бинарного дерева в массиве.....	6
3.5.1. Вставка.....	7
3.5.2. Поиск по индексу.....	7
3.5.3. Удаление.....	8
3.5.4. Балансировка.....	9
3.5.5. Итератор	10
3.5.6. Сохранение(загрузка) в(из) бинарный(ого) файл(а)	11
3.5 Пример работы разработанного ПО	11
Заключение	14
Приложение А	15

1 Задание на лабораторную работу

Портировать из Java разработанный в л.р.1 программный код.

2 Исходные данные

Согласно варианту задания на лабораторную работу имеем:

- Структура данных: бинарное дерево на основе массива;
- Типы данных: Целочисленный, Дата-Время.

3 Ход работы

3.1 Типы данных

Целочисленный тип данных реализован в классе *IntegerClass*. Данный класс содержит в себе единственное поле – значение типа *int* (целочисленный тип). Реализованы геттеры и сеттеры, переопределён метод *toString()*.

Тип Дата-Время более комплексный и реализован в классе *DateTimeClass*. Содержит в себе поля: день, месяц, год, час, минута, секунда. Все значения полей класса - целочисленные. Реализованы проверки на установку настоящей даты (проверка на високосный год и количество дней в месяце, принадлежность значения заданному интервалу). Реализованы геттеры и сеттеры, переопределён метод *toString()*.

Код реализации типов данных на языке программирования *Kotlin* представлен в Приложении А.

3.2 Паттерны/шаблоны проектирования

3.3.1. Фабрика (Factory)

В лабораторной работе №1 был реализован шаблон проектирования «Фабрика». В лабораторной работе №3 был портирован программный код данного шаблона проектирования из *Java* в *Kotlin*. Суть фабрики сводится к тому, чтобы производить объекты разного типа. В методе *getBuilderByName(name: String): ProtoType* фабрика получает на вход

строку с названием выбранного типа данных, а возвращает объект-прототип.

```
class FactoryType {
    private var typeNameList : List<ProtoType> =
listOf(IntegerType(), DateTimeType())

    fun getTypeNameList(): List<String?>{
        var listOfType: List<String?> = listOf()
        for (t in typeNameList){
            listOfType += t.typeName()
        }
        return listOfType
    }

    fun getBuilderByName(name: String): ProtoType{
        if (name == null) throw NullPointerException()
        for (userType in typeNameList){
            if (name == userType.typeName())
                return userType
        }
        throw IllegalArgumentException()
    }
}
```

Листинг 1. Реализация шаблона проектирования «Фабрика» на Kotlin

3.3.2. Объект-прототип/Строитель (Prototype/Builder)

В лабораторной работе №1 был реализован шаблон проектирования «Объект-прототип/Строитель». В лабораторной работе №3 был портирован программный код данного шаблона проектирования из *Java* в *Kotlin*. Суть его в том, чтобы обеспечить дополнительную обёртку для классов типов данных. Для реализации данного паттерна были переопределены для каждого типа данных все стандартные методы, а именно: *typeName()*, *create()*, *clone()*, *readValue()*, *parseValue()*, *getTypeComparator()*, *toString()*. Назначение каждого метода описано в комментариях к реализации шаблона проектирования (листинг 2). В конечном итоге были реализованы ещё два класса, наследующие типаж (trait) *ProtoType*: *IntegerType* и *DateTimeType*.

Код реализации этих классов на языке программирования *Kotlin* приведён в Приложении А.

```
interface ProtoType {
    // Имя типа
    fun typeName(): String?

    // Создание объекта
    fun create(): Any?

    // Клонирование текущего
    fun clone(obj: Any?): Any?

    // Создание и чтения объекта
    fun readValue(inputStream: InputStream?): Any?

    // Создает и парсит содержимое из строки
    fun parseValue(someString: String?): Any?

    // Возврат компаратора для сравнения
    val typeComparator: Comparator?

    // Перевод объекта в строку
    fun toString(`object`: Any): String
}
```

Листинг 2. Интерфейс для шаблона «Объект-прототип/Строитель»

3.3 Интерфейсы

Помимо уже рассмотренного интерфейса *ProtoType*, был реализован интерфейс компаратора *Comparator* (листинг 3). Данный типаж содержит в себе только один метод для переопределения – *compare(o1: Any?, o2: Any?): Int*, который позволяет сравнить значения соответствующего типа данных. Были реализованы два класса наследующий данный типаж: *IntegerComparator* (используется для сравнения целочисленных значений) и *DateTimeComparator* (используется для сравнений значений типа данных Дата-Время). Код реализации этих классов на языке программирования *Kotlin* приведён в Приложении А.

```
interface Comparator {  
    fun compare(o1: Any?, o2: Any?): Int  
}
```

Листинг 3. Интерфейс для реализации компаратора

3.4 Структура бинарного дерева в массиве

Структура бинарного дерева была подробно рассмотрена в лабораторной работе №1.

Класс *BinaryTreeArray* – бинарного дерева в массиве содержит следующие поля:

- *var arrayTree: ArrayList<AnyRef?>?* – исходный динамический массив, который хранит объекты – узлы дерева;
- *var size: Int* – поле, обозначающее размер массива;
- *var comparator: Comparator* – компаратор, он предназначен для сравнения объектов внутри методов класса.

Для работы со структурой реализованы следующие методы на языке программирования *Kotlin*:

- *save(protoType: ProtoType?)* – запись объектов дерева в бинарный файл;
- *load(protoType: ProtoType?): BinaryTreeArray?* – чтение объектов дерева из бинарного файла;
- *addValue(value: Any)* – добавление объекта в дерево;
- *printTree()* – печать дерева в консоль;
- *printArray()* – печать массива в консоль;
- *getDataAtIndex(searchIndex: Int): Any* – получение значения по индексу;
- *removeNodeByIndex(index: Int)* – удаление по индексу;
- *forEach(func: DoWith)* – итератор;
- *balance: BinaryTreeArray* – балансировка дерева;
- *toString* – преобразование дерева в строку (необходимо для GUI).

Код реализации класса *BinaryTreeArray* на языке программирования *Kotlin* приведён в Приложении А.

Рассмотрим основные алгоритмы данной структуры.

3.5.1. Вставка

Включение нового значения в двоичное дерево также базируется на ветвлении: при сравнении включаемого значения со значением в текущей вершине выбирается правая или левая ветвь до тех пор, пока не встретится свободная (листинг 3). При этом, если индекс вставки будет больше, чем размер массива, то массив необходимо увеличить.

```
// Вспомогательный метод вставки значения в массив
private fun insertRecursive(current: Int, obj: Any) {
    if (current >= size) { // увеличение размерности при
        выходе
        size *= 2 // за пределы массива
        for (i in size / 2..size) // с обнулением новой
        части
            arrayTree!!.add(null)
    }
    if (arrayTree!![current] == null) {
        arrayTree!![current] = obj
        return
    }
    if (comparator!!.compare(obj, arrayTree!![current]) <
0) insertRecursive(
        2 * current + 1,
        obj
    ) else insertRecursive(2 * current + 2, obj)
}

// Вставка значения в дерево
fun addValue(value: Any) {
    insertRecursive(0, value)
}
```

Листинг 3. Метод вставки нового значения в дерево

3.5.2. Поиск по индексу

Поиск заданного значения в двоичном дереве использует линейно рекурсивный алгоритм. В каждой вершине производится сравнение

размера левого поддерева с искомым номером. Если искомый номер меньше, чем размер левого поддерева, то рекурсивно ищется в левом поддереве. Если нет – то от искомого индекса вычитаются число вершин левого поддерева. Далее если декремент получившейся величины совпадает с нулём, то элемент найден, а если нет, то ищется индекс в правом поддереве.

В лабораторных работах №1 и №2 реализована индексация вершин по логическому номеру начиная с нуля «слева-направо» (листинг 4).

```
// Число вершин в поддереве
private fun getSize(num: Int): Int {
    return if (num >= size || arrayTree!![num] == null) 0 else
    1 + getSize(2 * num + 1) + getSize(2 * num + 2)
}

private fun getDataAtIndexRecursive(searchIndex: Int, help:
Int): Any? {
    var searchIndex = searchIndex
    if (searchIndex >= size || searchIndex >= getSize(help))
return null
    val cntL = getSize(2 * help + 1) // число вершин в левом
поддереве
    if (searchIndex < cntL) return getDataAtIndexRecursive(
        searchIndex,
        2 * help + 1
    ) // Логический номер в левом поддереве
    searchIndex -= cntL // отбросить вершины левого поддерева
    return if (searchIndex-- == 0) arrayTree!![help] else
getDataAtIndexRecursive(
    searchIndex,
    2 * help + 2
    ) // Логический номер – номер текущей вершины
    // в правое поддерево с остатком Логического номера
}

//нумерация "слева-направо", начинается с 0, см. cprog 8.5
fun getDataAtIndex(searchIndex: Int): Any? {
    return getDataAtIndexRecursive(searchIndex, 0)
}
```

Листинг 4. Метод поиска значения в дереве

3.5.3. Удаление

В первую очередь происходит поиск удаляемого элемента, после нахождения которого следует этап, у которого могут быть три вариации:

1 Удаляемый узел является листовым (не имеет потомков).

В данном случае просто удаляем лист из дерева.

2 Удаляемый узел имеет одного потомка.

В таком случае мы удаляем выбранный узел, и на его место ставим его потомка. После это сдвигаем поддеревья переставляемого узла в массиве так, чтобы сохранились связи.

3 Удаляемый узел имеет двух потомков.

В данном случае, необходимо в левом поддереве найти максимальный элемент (он же будет листом) и переставить его на место удаляемого узла.

Так как реализация удаления из дерева на языке программирования *Kotlin* вышла достаточно объёмная, то ознакомиться с ней можно в Приложении А с подробными комментариями.

3.5.4. Балансировка

Реализована рекурсивная балансировка на языке программирования *Kotlin* – выполняется обход с сохранением упорядоченной последовательности массива, при котором индексы узлов собираются в *Vector*. Затем *Vector* рекурсивно делится пополам, значение из середины включается в новое дерево. Итоговое дерево получается сбалансированным (*cprog*, раздел 8.5). Метод возвращает новый объект *BinaryTreeArray* (листинг 5).

```
//рекурсивная балансировка
private fun balance(t: Vector<Any?>, a: Int, b: Int, r:
ArrayList<Any?>) {
    if (a > b) return
    if (a == b) return
    val m = a + b ushr 1 // взять строку из середины интервала
    insertRecursive(r, 0, t[m])
    balance(t, m + 1, b, r) // рекурсивно выполнить для левой
и
    balance(t, a, m, r) // правой частей
}

//вставка для нового аррайлист при балансировке
```

```

private fun insertRecursive(t: ArrayList<Any?>, current: Int,
obj: Any?) {
    if (current >= size) { // увеличение размерности при
выходе
        size *= 2 // за пределы массива
        for (i in size / 2..size) // с обнулением новой части
            t.add(null)
        }
    if (t[current] == null) {
        t[current] = obj
        return
    }
    if (comparator!!.compare(obj, t[current]) < 0)
insertRecursive(t, 2 * current + 1, obj) else insertRecursive(
    t,
    2 * current + 2,
    obj
)
}

//главный метод балансировки
fun balance(): BinaryTreeArray {
    val sz1 = getSize(0)
    val newArray = Vector<Any?>(size) //вектор индексов
    val newArrayTree = ArrayList<Any?>(size)
    for (i in 0 until size) {
        newArrayTree.add(null)
    }
    set(newArray, 0)
    balance(newArray, 0, sz1, newArrayTree)
    return BinaryTreeArray(size, newArrayTree, comparator)
}

//метод для добавления индексов в вектор
private operator fun set(t: Vector<Any?>, n: Int) {
    if (n >= size || arrayTree!![n] == null) return
    set(t, 2 * n + 1)
    t.add(arrayTree!![n])
    set(t, 2 * n + 2)
}

```

Листинг 5. Балансировка дерева

3.5.5. Итератор

Под итератором *forEach* понимается метод (листинг 6), который проходит структуру данных линейно и вызывает переданную лямбду (листинг 7) для каждого хранимого элемента данных, а если лямбда срабатывает, то возвращает.

```

// итератор forEach
fun forEach(func: DoWith) {
    if (arrayTree == null || size <= 0) return
    val sz = getSize(0)
    val v = Vector<Int>(size)
    setHelp(v, 0)
    for (i in 0 until sz) {
        func.doWith(arrayTree!![v[i]])
    }
}

```

Листинг 6. Реализация итератора

```

interface DoWith {
    fun doWith(obj: Any?)
}

```

Листинг 7. Интерфейс для реализации функционально тривиальных операций над элементами списка структуры данных

3.5.6. Сохранение(загрузка) в(из) бинарный(ого) файл(а)

В классе структуры данных также имеются методы сохранения дерева в двоичный файл и загрузки его из двоичного файла (листинг 8). Сериализация и десериализация представляет собой сохранение и загрузку хранимых объектов дерева, а не самого дерева.

```

fun save(protoType: ProtoType?) {
    val writer = BufferedWriter(Writer("saved"))
    if (protoType != null) {
        writer.write(protoType.typeName() + "\n")
    }
    this.forEach(object : DoWith {
        override fun doWith(obj: Any?) {
            val x = obj
            if (protoType != null) {
                writer.write(
                    """
                                ${protoType.toString(x!!)}
                                """.trimIndent()
                )
            }
        }
    })
    if (writer != null) writer.close()
}

fun load(protoType: ProtoType?): BinaryTreeArray? {
    val fileName = "saved"
    val bts = BinaryTreeArray(this.comparator)
    try {

```

```

        BufferedReader(FileReader(fileName)).use { br ->
            var line: String?
            line = br.readLine()
            if (protoType != null) {
                if (!protoType.typeName().equals(line)) {
                    throw Exception("Wrong file
structure")
                }
            }
            while (br.readLine().also { line = it } !=
null) {
                if (protoType != null) {
bts.addValue(protoType.parseValue(line!!))
                }
            }
        } catch (e: Exception) {
            throw RuntimeException(e)
        }
        return bts;
    }
}

```

Листинг 8. Реализация сохранения в файл и загрузки из файла

3.5 Пример работы разработанного ПО

В *Main* были написаны все методы по работе с деревом для демонстрации. Результат их работы приведён на рис. 1-4.

Также в ходе лабораторной работы был реализован *GUI* (рис. 5).

Код реализации класса *GUI* приведён в Приложении А.

```

-----TEST FOR INTEGER-----
-----PRINT TREE-----
      -940
        -856
      -853
    -273
    -183
585
  668
-----PRINT ARRAY-----
-940 -856 -853 -273 -183 585 668
----GET VALUE BY INDEX 2----
value = -853
---DELETE VALUE BY INDEX 2---
      -940
        -856
      -273
        -183
585
  668

```

Рисунок 1. Демонстрация работы операций над деревом при ТД
“IntegerClass”

```

-----SAVE IN BINARY FILE-----
-----BALANCE-----
      -940
    -856
    -273
  -183
    585
    668
---LOAD FROM BINARY FILE---
-940
  -856
    -273
      -183
        585
        668

-----FOR EACH-----
-940
-856
-273
-183
585
668

```

Рисунок 2. Демонстрация работы операций над деревом при ТД
“IntegerClass”

```

-----TEST FOR DATETIME-----
-----PRINT TREE-----
  3/3/27 08:46:13
3/8/329 20:58:47
  12/4/373 12:05:41
    25/5/381 07:57:54
  15/3/767 20:30:15
    21/8/768 01:11:36
      30/5/1050 05:41:25
-----PRINT ARRAY-----
3/3/27 08:46:13 3/8/329 20:58:47 12/4/373 12:05:41 25/5/381 07:57:54 15/3/767 20:30:15 21/8/768 01:11:36 30/5/1050 05:41:25
----GET VALUE BY INDEX 2----
value = 12/4/373 12:05:41
---DELETE VALUE BY INDEX 2---
  3/3/27 08:46:13
3/8/329 20:58:47
  25/5/381 07:57:54
  15/3/767 20:30:15
    21/8/768 01:11:36
      30/5/1050 05:41:25
-----SAVE IN BINARY FILE-----

```

Рисунок 3. Демонстрация работы операций над деревом при ТД
“DateTimeClass”

```

-----SAVE IN BINARY FILE-----
-----BALANCE-----
      3/3/27 08:46:13
    3/8/329 20:58:47
      25/5/381 07:57:54
15/3/767 20:30:15
      21/8/768 01:11:36
    30/5/1050 05:41:25
---LOAD FROM BINARY FILE---
3/3/27 08:46:13
  3/8/329 20:58:47
    25/5/381 07:57:54
      15/3/767 20:30:15
        21/8/768 01:11:36
          30/5/1050 05:41:25

-----FOR EACH-----
3/3/27 08:46:13
3/8/329 20:58:47
25/5/381 07:57:54
15/3/767 20:30:15
21/8/768 01:11:36
30/5/1050 05:41:25

```

Рисунок 4. Демонстрация работы операций над деревом при ТД
“DateTimeClass”

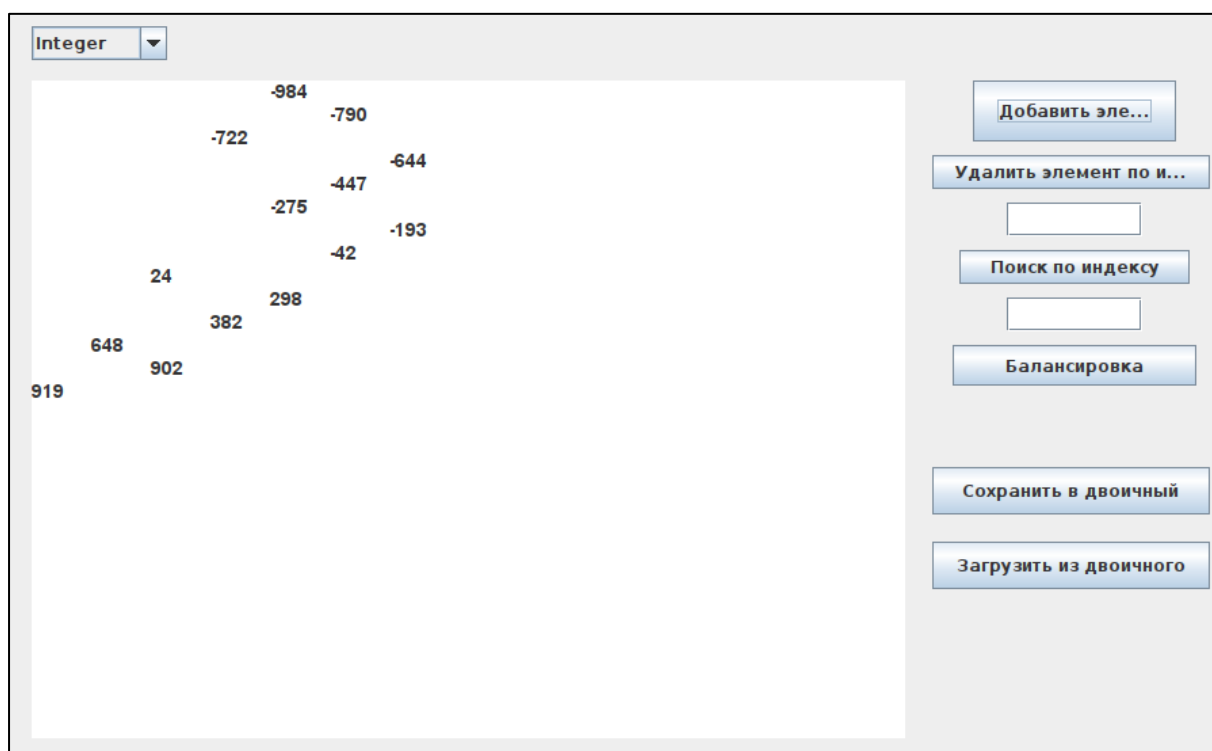


Рисунок 5. Демонстрация работы GUI

Заключение

В ходе выполнения лабораторной работы было разработано и протестировано программное обеспечение, позволяющее работать со структурой данных «Бинарное дерево в массиве» с различными типами данных (целочисленный тип и дата-время). Были применены шаблоны/паттерны проектирования «Фабрика» и «Объект-прототип/Строитель».

Программный код данного ПО был портирован с языка *Java* на *Kotlin*. Был сделан вывод, что возможности языка программирования *Kotlin* для разработки программного обеспечения являются достаточно удобными для написания кода.

Приложение А

Весь исходный код расположен по ссылке: